

Objetivos da Aula

- Compreender a organização e o papel dos **clientes** em sistemas distribuídos.
- Explorar diferentes abordagens para **interfaces de usuário** em rede.
- Analisar questões de projeto e arquiteturas comuns para **servidores**.
- Introduzir o conceito e as motivações para **migração de código**.
- Discutir modelos e desafios da migração de código em sistemas heterogêneos.

Cientes

O Papel do Cliente

Definição

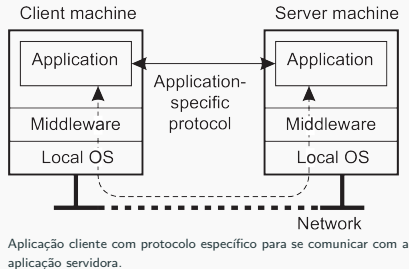
- Clientes são processos que requisitam serviços de servidores.
-
- Principal tarefa: Fornecer uma interface para o usuário interagir com serviços remotos.
 - Executam em máquinas separadas dos servidores (geralmente).
 - Software cliente pode incluir:
 - Interface de usuário.
 - Lógica de aplicação (parcial).
 - Componentes para transparência de distribuição.

3.3.1 Interfaces de Usuário em Rede

Duas abordagens principais para interação cliente-servidor via UI:

1. Protocolo Específico da Aplicação:

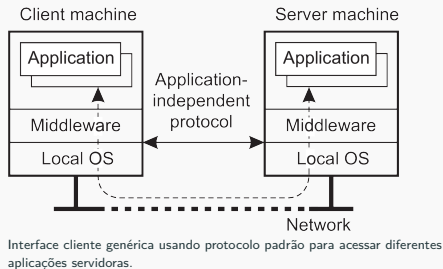
- Cada serviço remoto tem um programa cliente correspondente.
- Ex: Cliente de e-mail, cliente de calendário sincronizando com servidor.
- *Desvantagem:* Requer software cliente separado para cada serviço.



3.3.1 Interfaces de Usuário em Rede (cont.)

2. Protocolo Independente da Aplicação (Thin Client):

- Interface de usuário genérica para acessar múltiplos serviços remotos.
- Processamento e armazenamento ocorrem principalmente no servidor.
- Ex: Terminais X, Navegadores Web acessando aplicações remotas.
- *Vantagem:* Gerenciamento simplificado.



3.3.1 Interfaces de Usuário em Rede (cont.)

Exemplo: O Sistema X Window

- Um dos mais antigos sistemas de UI em rede.
- Separa a aplicação (cliente X) do servidor de display (kernel X).
- O **Kernel X** (no terminal do usuário) gerencia display, teclado, mouse.
- A **Aplicação** (pode ser remota) envia comandos de desenho e recebe eventos via **Protocolo X** (sobre rede).
- *Paradoxo*: O servidor X está na máquina do usuário; as aplicações são clientes X.
- Desafios: Latência de rede, sincronia do protocolo X.
- Soluções como VNC (Virtual Network Computing) controlam remotamente o desktop a nível de pixel.

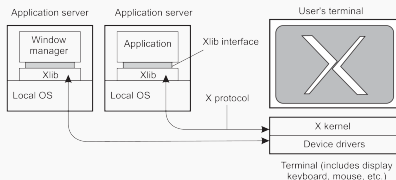


Ilustração da arquitetura (Kernel X, Xlib, Aplicação, Gerenciador de Janelas).

3.3.2 Ambiente de Desktop Virtual (VDE)

- Evolução do conceito de *thin client*, impulsionado pela computação em nuvem.
- O navegador web torna-se a interface primária para muitas aplicações.
- **Chrome OS:** Exemplo de sistema operacional centrado no navegador.
- **Anatomia de um Navegador Web (Ex: Chrome):**

- Extremamente complexo (> 25 milhões de linhas de código).

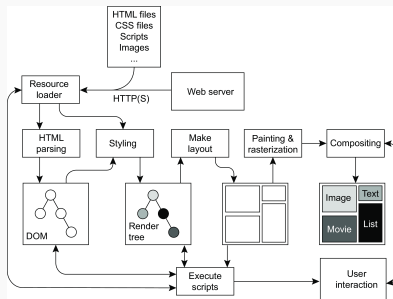


Ilustração do fluxo de renderização complexo.

- **Anatomia de um Navegador Web (Ex: Chrome):** (cont.)
- Múltiplos componentes: Carregador de recursos, parser HTML, motor de estilo, layout, pintura, rasterização, compositor, motor JavaScript/WebAssembly.
- Usa múltiplos processos e threads para performance e isolamento (sandboxing).
- **PWAs (Progressive Web Apps):** Aplicações web que funcionam como apps nativos, com capacidade offline (cache local).
- Gerenciamento facilitado: Atualizações são feitas no servidor/nuvem.

3.3.3 Software Cliente para Transparência

- Além da UI, o software cliente implementa mecanismos para esconder a distribuição.
- **Transparência de Acesso:**
 - Gerada por *stubs* de cliente (ex: RPC, RMI).
 - Mascaram diferenças de arquitetura, formatos de dados e comunicação de rede.
- **Transparência de Localização, Migração, Relocação:**
 - Utiliza sistemas de nomes.
 - Middleware cliente pode re-ligar (rebind) transparentemente a um servidor que mudou de local.
- **Transparência de Replicação:**
 - Software cliente pode contactar múltiplas réplicas, coletar respostas e apresentar uma única resposta. (Conceito da *Figura 3.18*).
- **Transparência de Falha:**

3.3.3 Software Cliente para Transparência (cont.)

- Middleware cliente pode tentar reconectar, contactar outra réplica, ou usar dados de cache.
- **Transparência de Concorrência:** Menos suportada no cliente, geralmente delegada a servidores intermediários (ex: monitores de transação).

Servidores

Definição

- Servidor: Um processo implementando um serviço específico para clientes.
- Organização básica:
 1. Espera por uma requisição de entrada.
 2. Processa a requisição.
 3. (Opcional) Envia uma resposta de volta.
 4. Volta ao passo 1.

3.4.1 Questões Gerais de Projeto

Servidores Concorrentes vs. Iterativos:

- **Iterativo:** Lida com uma requisição por vez, do início ao fim. Simples, mas pode bloquear enquanto processa.
- **Concorrente:** Passa a requisição para outra thread ou processo. Pode atender múltiplos clientes “simultaneamente”.
 - Ex: Servidor multithreaded (modelo dispatcher/worker), servidor que faz `fork()` (Unix).

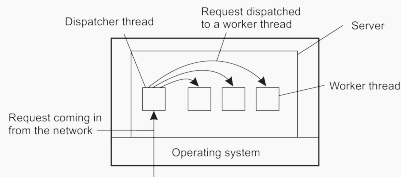
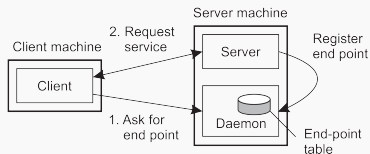


Ilustração do modelo Dispatcher/Worker Multithread.

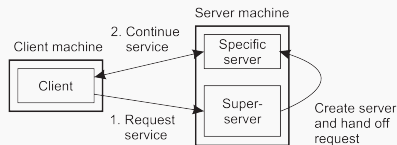
3.4.1 Questões Gerais de Projeto (cont.)

Contato com o Servidor: Endpoints:

- Clientes enviam requisições para um **endpoint** (ou **porta**) na máquina servidora.
- **Endpoints bem-conhecidos:** Atribuídos globalmente pela IANA (ex: TCP 80 para HTTP).
- **Endpoints dinâmicos:** Alocados pelo SO local. Requer um serviço de descoberta.
 - **Daemon:** Processo especial que mapeia serviços para endpoints atuais.
 - **Superserver (ex: inetd):** Único processo ouvindo múltiplas portas; inicia o servidor real sob demanda.



Daemon: Processo especial que mapeia serviços para endpoints atuais



Superserver: Único processo ouvindo múltiplas portas; inicia o servidor real sob demanda

3.4.1 Questões Gerais de Projeto

Interrupção de um Servidor:

- Como um cliente cancela uma operação longa (ex: upload)?
- Saída abrupta do cliente (quebra a conexão).
- **Dados fora-de-banda (Out-of-band):** Dados prioritários enviados pelo mesmo canal (ex: TCP URGENT) ou por um canal de controle separado para interromper o fluxo normal.

Servidores Stateless vs. Stateful:

- **Stateless:** Não mantém informação sobre o estado do cliente entre requisições. Ex: Servidor Web básico.
 - Vantagem: Recuperação simples após falha.
 - Pode usar *soft state* (estado temporário com expiração) ou *cookies* (estado armazenado no cliente).

3.4.1 Questões Gerais de Projeto (cont.)

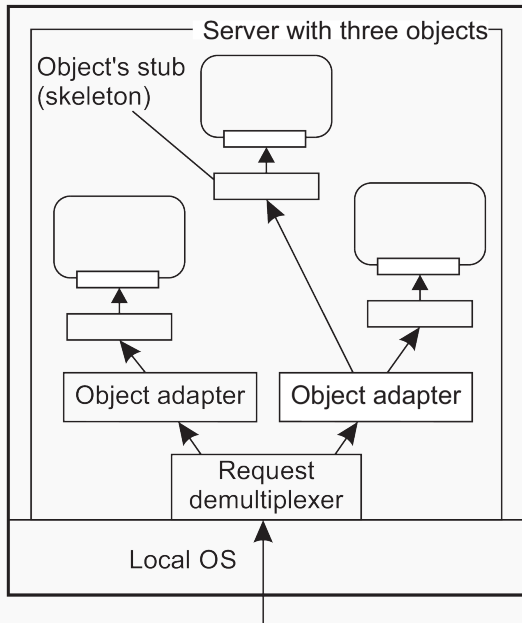
- **Stateful:** Mantém informação persistente sobre clientes. Ex: Servidor de arquivos permitindo cache local com escritas.
 - Vantagem: Potencialmente melhor performance (evita reenviar estado).
 - Desvantagem: Recuperação complexa após falha; precisa restaurar o estado.
 - Distinção entre *estado de sessão* (temporário) e *estado permanente*.

3.4.2 Servidores de Objeto

- Servidor cujo propósito é hospedar *objetos* distribuídos.
- O servidor em si não provê o serviço; os objetos hospedados o fazem.
- **Servant:** Peça de código que implementa um objeto.
- Flexibilidade: Serviços podem ser alterados adicionando/removendo objetos.

Políticas de Ativação: Como invocar um objeto?

1. **Objetos Transientes:** Existem apenas enquanto o servidor existe (ou menos). Criados sob demanda (na primeira invocação) ou na inicialização do servidor.
2. **Compartilhamento de Código/Dados:** Objetos da mesma classe podem compartilhar código. Objetos podem ter segmentos de memória separados ou compartilhados.
3. **Threading:** Uma thread por servidor, uma por objeto, uma por invocação, pool de threads.
4. **Object Adapter (ou Wrapper):** Componente genérico que implementa uma política de ativação específica. Despacha requisições para o *servant* correto via *skeleton* (stub do lado do servidor). Veja figura no próx slide.



3.4.3 Exemplo: Apache Web Server

- Altamente configurável, extensível e portátil.
- **APR (Apache Portable Runtime):** Biblioteca que abstrai o SO (arquivos, rede, threads, etc.).
- Arquitetura modular: Funcionalidade adicionada via **módulos**.
- **Hooks:** Pontos no fluxo de processamento onde funções de módulos podem ser chamadas. Ex: Tradução de URL, verificação de acesso, log, tipo MIME.
- **Fases:** Controlam a ordem em que hooks são processados (início, meio, fim).
- Design favorece módulos stateless e isolados.

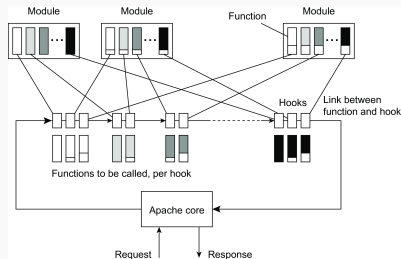
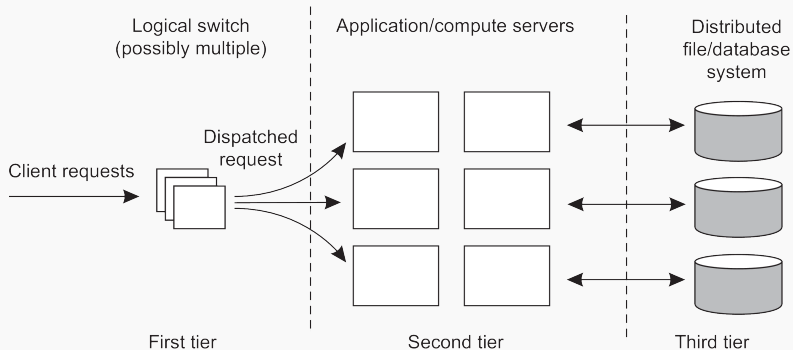


Ilustração da organização geral do Apache com módulos e hooks.

3.4.4 Clusters de Servidores

- Coleção de máquinas (geralmente em rede local) rodando servidores.
- Objetivo: Escalabilidade e/ou alta disponibilidade.

Organização Geral (3 Tiers)

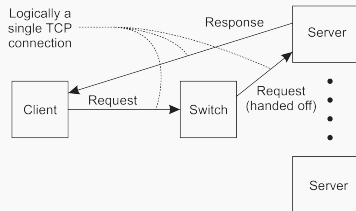


1. Primeiro Tier (Front End / Switch):

- Ponto único de acesso.
- Roteia requisições para os servidores apropriados.
- Pode ser **switch de camada de transporte** (baseado em TCP/IP, ex: round-robin) ou **switch de camada de aplicação** (inspeciona conteúdo, ex: URL, tipo de requisição).
- Ex: Nginx como *reverse proxy*.
- **TCP Handoff**: Otimização onde o switch passa a conexão TCP diretamente para o servidor.

2. Segundo Tier (Servidores de Aplicação/Computação):

- Executam a lógica da aplicação.
- Podem ser especializados (ex: processamento de vídeo).



3. Terceiro Tier (Servidores de Dados):

- Gerenciam armazenamento (arquivos, bancos de dados).
- Podem ser otimizados para I/O.

3.4.4 Clusters de Servidores (cont.)

Clusters em Wide-Area:

- Servidores distribuídos geograficamente.
- **CDNs (Content Delivery Networks):**

Exemplo proeminente.

- Objetivo: Proximidade ao cliente (baixa latência, alta largura de banda).
- Usa redirecionamento (DNS, HTTP) para direcionar clientes ao servidor de borda (edge server) mais próximo/melhor.
- Replicação de conteúdo sob demanda.

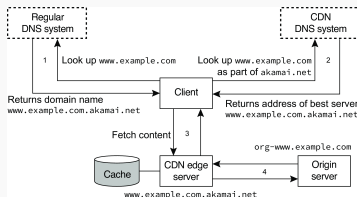


Ilustração da arquitetura da CDN Akamai

Migração de Código



Definição

- Não apenas passar dados, mas também mover programas (código) entre máquinas.
- Pode incluir o estado de execução (migração de processo).

3.5.1 Razões para Migrar Código

1. Performance:

- **Balanceamento de Carga:** Mover processos de máquinas sobrecarregadas para ociosas (tradicionalmente).
- **Otimização de Energia:** Mover VMs para consolidar trabalho em menos máquinas físicas (data centers).
- **Redução de Comunicação:** Mover a computação para perto dos dados.
 - Cliente envia parte da aplicação para o servidor de BD.
 - Servidor envia código de validação de formulário para o cliente.
 - *Offloading* de computação para smartphones.
- **Paralelismo:** Agentes móveis que buscam informações em paralelo.

2. Flexibilidade:

- Configuração dinâmica de sistemas distribuídos.
- Cliente baixa a implementação do lado cliente de um serviço sob demanda, quando se conecta ao servidor.
- Ex: JavaScript em páginas web, Java Applets (histórico).
- *Vantagem:* Cliente não precisa ter todo o software pré-instalado; protocolo pode evoluir.
- *Desvantagem:* Segurança (código malicioso).

3. Privacidade/Segurança:

- Manter dados sensíveis locais e trazer o modelo/código para perto dos dados.
- **Aprendizado Federado (Federated Learning):** Treinar modelos de ML localmente em dispositivos/organizações sem compartilhar os dados brutos. O servidor central agrega os modelos atualizados; figura próx slide.

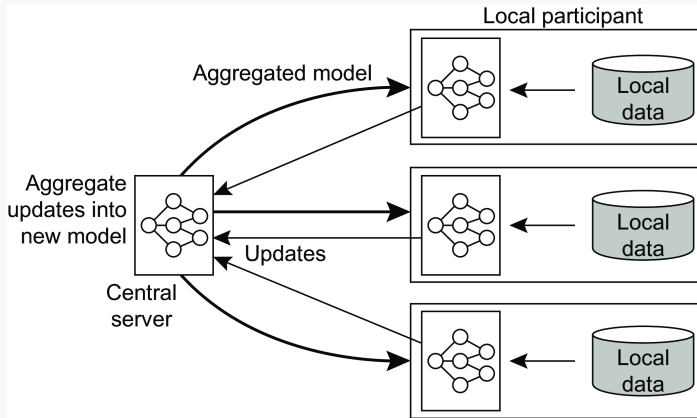


Ilustração do modelo de treinamento de ML Federated Learning

3.5.2 Modelos para Migração de Código

Framework de Fuggetta et al. [1998]: Processo = (Segmento de Código, Segmento de Recursos, Segmento de Execução)

- **Segmento de Código:** As instruções do programa.
- **Segmento de Recursos:** Referências a recursos externos (arquivos, dispositivos, etc.).
- **Segmento de Execução:** Estado atual (pilha, dados privados, contador de programa).

Mobilidade Fraca vs. Forte:

- **Mobilidade Fraca:** Apenas o segmento de código (e dados de inicialização) é transferido. O programa sempre inicia do zero no destino.
Ex: Java Applets.
 - Simples, foca na portabilidade do código.

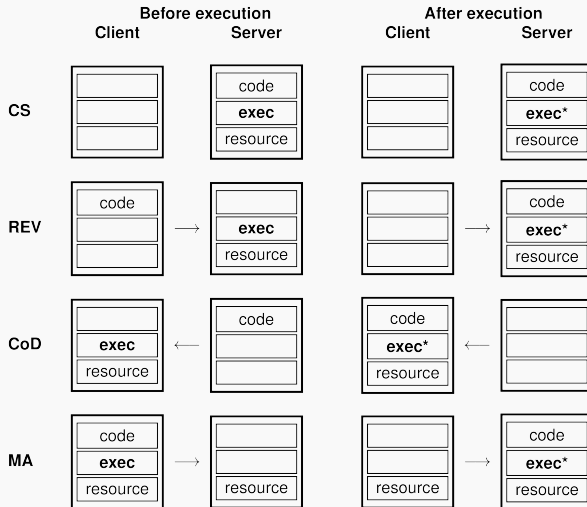
3.5.2 Modelos para Migração de Código (cont.)

- **Mobilidade Forte:** Também transfere o segmento de execução. O processo é pausado, movido e continua de onde parou. Ex: Migração de processo.
 - Mais geral, porém mais complexa (dependência do estado do SO).
 - **Clonagem Remota:** Cria uma cópia exata do processo em outra máquina, que executa em paralelo (similar ao `fork()`).

3.5.2 Modelos para Migração de Código (cont.)

Paradigmas:

- **Cliente-Servidor (CS):** Código e estado no servidor. Cliente apenas envia requisições. Nenhuma migração real.
- **Avaliação Remota (REV):** Iniciada pelo remetente. Código migra do cliente para o servidor e executa lá.
- **Código sob Demanda (CoD):** Iniciada pelo receptor. Cliente baixa código do servidor e executa localmente.
- **Agente Móvel (MA):** Iniciado pelo remetente. Código e estado de execução migram (potencialmente por múltiplas máquinas).



CS: Client-Server
CoD: Code-on-demand

REV: Remote evaluation
MA: Mobile agents

Paradigmas de migração de código

3.5.3 Migração em Sistemas Heterogêneos

- Desafio: Diferentes arquiteturas de máquina e sistemas operacionais.
- Solução: **Máquinas Virtuais (VMs)**.
 - **VMs de Processo:** Abstraem o SO e hardware para uma única linguagem/aplicação. Ex: JVM (Java Virtual Machine), interpretadores de script (Python). Permitem mobilidade fraca (portabilidade de código).
 - **VMs de Sistema (Monitores de VM / Hipervisores):** Emulam o hardware subjacente. Permitem que um SO convidado completo (guest OS) e suas aplicações rodem em cima de um SO hospedeiro (host OS) ou diretamente no hardware (nativo).
 - Facilitam a **migração de ambientes computacionais inteiros**.
 - Crucial para computação em nuvem e data centers.
 - Técnicas de migração de VM (live migration): pre-copy, stop-and-copy, pull (on-demand).

- **Clientes** fornecem interfaces e usam software para transparência distribuída.
- **Servidores** têm vários designs (iterativo/concorrente, stateful/stateless) e podem ser organizados em clusters para escalabilidade/disponibilidade.
- **Migração de Código** move programas para otimizar performance, flexibilidade ou segurança/privacidade.
- Diferentes **modelos de migração** existem, com trade-offs entre complexidade e capacidade (mobilidade fraca vs. forte).
- **VMs** são fundamentais para migração em ambientes heterogêneos.

That's all Folks!